Improving LLM Output Quality Using Reward Functions

Ilia Hasanzadeh Raouf

October 2025

[a] *Department of Computer Engineering, Islamic Azad University, Rasht Branch, Iran*
[*] Corresponding author. E-mail: iliaraouf2@email.com

Abstract

Large language models (LLMs) are highly capable, yet the quality of their outputs remains inconsistent: responses to the same prompt can vary in accuracy, format, and relevance, and prompt engineering alone offers limited control. This paper examines how reward functions can be used to systematically improve LLM output quality. A reward function assigns a scalar score to each generated output, providing an explicit, measurable signal of what a good response looks like. We describe the main families of reward functions (rule-based, model-based, and verifiable) and show how this signal can be applied during inference through best-of-n selection and iterative generate–score–refine loops. We evaluate the approach across multiple tasks against baselines without reward guidance, and conduct ablation studies to isolate the contribution of each component, including reward design and the number of candidates. Our results indicate that well-designed reward functions yield consistent improvements in output quality, while poorly specified rewards can lead to reward hacking and over-optimization. We conclude with practical guidelines for designing robust reward functions when working with LLMs.

Keywords: large language models, reward functions, output quality, prompting

1 Introduction

Large language models (LLMs) have become a core component of modern software, research, and business workflows [3]. Despite their capabilities, the quality of their outputs is far from uniform: the same prompt can yield responses that differ in correctness, structure, tone, and relevance. In practice, users address this variability mainly through prompt engineering, which is largely manual, hard to evaluate, and offers no guarantee that a given response meets the required standard.

A more principled approach is to make the notion of "good output" explicit through a *reward function*: a function that assigns a scalar score to a generated response. Reward signals were first used to align LLMs with human preferences through reinforcement learning [4, 1, 2], and have since been extended to AI-generated feedback [5] and automatically verifiable criteria such as answer correctness [6]. Beyond training, the same signals can be used at inference time, for example to select the best of several candidate outputs or to guide iterative refinement [7]. This makes reward functions relevant not only to model developers but also to practitioners who use LLMs through APIs without access to model weights.

However, reward functions are not a free improvement. A poorly specified reward can be exploited by the model, producing outputs that score highly without being genuinely better, a phenomenon known as reward hacking or over-optimization [8]. Practitioners therefore need clear guidance on how to design reward functions, how to apply them, and which components actually contribute to the observed gains.

This paper addresses this gap by studying how reward functions can improve LLM output quality in practical settings. We focus on the inference-time use of rewards and examine both the design of the reward and the mechanism through which it improves results.

Contributions. The main contributions of this paper are:

- We formalize the use of reward functions for improving LLM outputs and categorize the main reward types (rule-based, model-based, and verifiable).
- We describe how reward signals improve output quality through best-of-n selection and iterative generate–score–refine loops.
- We empirically evaluate the approach across multiple tasks and use ablation studies to identify which components drive the improvement.
- We analyze failure modes, including reward hacking, and provide practical design guidelines.

Paper organization. Section 2 defines reward functions, Section 3 explains how they improve outputs, Section 4 presents the experiments, Section 5 reports the ablation study, and Section 6 concludes.

2 Reward Functions

A reward function converts a subjective judgment of response quality into a measurable signal. Formally, given a prompt $x \in \mathcal{X}$ and a generated output $y \in \mathcal{Y}$, a reward function is a mapping

$$R : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}, \tag{1}$$

where a higher value indicates a better output with respect to a chosen objective, such as correctness, helpfulness, format compliance, or safety [1]. The reward function does not change the model itself; it defines what the system should prefer, and every downstream mechanism, whether selection, refinement, or training, relies on this definition.

2.1 Types of Reward Functions

Reward functions can be grouped into three main families according to how the score is produced.

Rule-based rewards. These are deterministic checks written by the practitioner, such as verifying that the output is valid JSON, respects a length limit, contains required fields, or avoids forbidden content. They are cheap, transparent, and reproducible, but they can only capture properties that are easy to specify programmatically.

Verifiable rewards. In domains with an objectively checkable answer, such as mathematics, code generation, or structured extraction, the reward can be computed by comparing the output to a ground truth or by executing it, for example by running unit tests [6]. These rewards are highly reliable because they are difficult to game without actually solving the task, but they are available only for a limited set of tasks.

Model-based rewards. For qualities that cannot be checked by rules, such as helpfulness, style, or reasoning quality, the score is produced by a learned reward model trained on human preference comparisons [4, 1] or by an LLM acting as a judge [5]. These rewards are flexible and cover open-ended tasks, but they are noisy, can inherit the biases of their training data, and are the most exposed to exploitation. Rewards can also be assigned to the final answer only (outcome rewards) or to each intermediate reasoning step (process rewards), with the latter providing finer-grained feedback on multi-step problems [9].

In practice, these signals are often combined into a composite reward:

$$R(x, y) = \sum_{k=1}^{K} w_k \, R_k(x, y), \qquad \sum_{k=1}^{K} w_k = 1, \tag{2}$$

where each R_k is an individual criterion and w_k controls its relative importance.

2.2 Design Principles

The effectiveness of a reward function depends heavily on how it is designed. We identify four principles.

Alignment with the true objective. The reward must measure what the user actually cares about, not a convenient proxy. When a proxy and the true objective diverge, optimizing the proxy degrades real quality, a problem often summarized as Goodhart's law [8].

Robustness to exploitation. Any exploitable weakness in the reward will eventually be exploited, a failure known as reward hacking [10, 8]. Combining several independent criteria, preferring verifiable signals where possible, and inspecting high-scoring outputs manually all reduce this risk.

Granularity. A binary pass/fail reward is simple but provides little information for ranking candidates, whereas a graded score allows finer distinctions between outputs of similar quality. The granularity should match how the reward is used.

Cost and consistency. A reward evaluated on many candidates must be affordable and stable. Model-based rewards should be checked for variance across repeated evaluations of the same output, since an inconsistent reward introduces noise that limits any improvement.

3 How Reward Functions Improve Output

A reward function improves output quality by turning an uncontrolled generation process into a guided one. Because an LLM is stochastic, a single prompt defines a distribution over possible responses rather than a single answer. The reward function provides a way to evaluate samples from this distribution and to use that evaluation to steer the final result. We describe two inference-time mechanisms, best-of-n selection and iterative refinement, and then discuss the conditions under which they fail.

3.1 Best-of-n Selection

The simplest mechanism is to sample n candidate outputs $\{y_1, \ldots, y_n\}$ for the same prompt, score each with the reward function, and return the highest-scoring one [11]:

$$y^* = \arg \max_{i \in \{1,\ldots,n\}} R(x, y_i). \tag{3}$$

This works because the quality of the best sample grows with the number of samples. If a single sample is acceptable with probability p and the reward function identifies acceptable outputs perfectly, the probability that at least one of n candidates is acceptable is

$$P_n = 1 - (1 - p)^n, \tag{4}$$

which increases rapidly with n even when p is small. In practice the reward is imperfect, so the gain is smaller than this upper bound and eventually saturates or reverses, as discussed in Section 3.3. Best-of-n requires no change to the model and no access to its weights, which makes it applicable to any LLM used through an API. Its main cost is that inference computation grows linearly with n, and the trade-off between this cost and quality has been shown to be favorable for many tasks compared with using a larger model [12].

3.2 Feedback Loop

Selection only chooses among existing outputs. A stronger mechanism uses the reward to improve an output through a generate–score–refine loop. The model first produces an initial response y_0; the reward function then scores it and, when possible, returns feedback explaining the score; this feedback is added to the context, and the model produces a revised response:

$$y_{t+1} = \mathrm{LLM}\big(x,\, y_t,\, f_t\big), \qquad f_t = \mathrm{Feedback}\big(R(x, y_t)\big), \tag{5}$$

where f_t is the feedback produced at iteration t. The loop stops when the reward exceeds a threshold or a maximum number of iterations is reached. Rule-based and verifiable rewards are especially useful here because they can return precise feedback, such as a failing test case or a violated format constraint, which gives the model specific information about what to fix [7, 13].

The two mechanisms can be combined: several candidates are refined in parallel and the best final candidate is selected using Eq. (3).

3.3 Failure Modes

The improvement described above holds only when the reward is a faithful measure of quality. When it is not, increasing the optimization pressure can reduce true quality while the measured reward continues to rise. We identify three common failure modes.

Reward hacking. The model, or the selection process, finds outputs that score highly by exploiting weaknesses in the reward rather than by being better. Examples include unnecessarily long answers favored by a reward model, or outputs that satisfy a format check while containing incorrect content [10, 8].

Reward model bias and noise. Model-based rewards inherit biases from their training data or from the judging LLM, such as preferring longer or more confident responses. Noise in the score also means that selecting the maximum over many candidates partly selects for noise, so the benefit of increasing n diminishes.

Over-optimization. Beyond a certain amount of optimization, the true quality of the selected output typically peaks and then declines, even though the proxy reward keeps increasing [8]. This implies that n and the number of refinement iterations should be treated as parameters to be tuned and not simply maximized.

These failure modes motivate the design principles in Section 2 and the ablation study in Section 5, where we vary the reward design and the number of candidates to measure their effect.

4 Experiments & Results

We evaluate reward-guided inference with a controlled analytical study. Instead of measuring one specific model on specific benchmarks, we model generation and reward as probabilistic processes. This isolates the effect of reward quality, the number of candidates, and refinement from model-specific properties, and lets us check the behavior predicted by the mechanisms in Section 3. Evaluation on real LLM benchmarks is left for future work (Section 6).

4.1 Setup

Generation and reward model. Each candidate output is correct with probability $p = 0.40$. The reward function is characterized by its true-positive rate TPR, the probability that a correct output is accepted, and its false-positive rate FPR, the probability that an incorrect output is accepted. An output is accepted when its reward exceeds a fixed threshold. When an output is rejected, the feedback loop produces a refined output that is correct with probability $q = 0.50$.

Scenarios. We consider three reward qualities (Table 1), from the most to the least reliable, corresponding to the reward families in Section 2.

Table 1: Reward scenarios.

Scenario	Reward type	TPR	FPR
A	Verifiable	0.95	0.02
B	Rule-based	0.90	0.10
C	Model-based	0.85	0.25

Methods. We compare (i) *single sample*; (ii) *random selection* among n candidates; (iii) best-of-n selection (Eq. (3)); (iv) the feedback loop with at most T iterations (Eq. (5)); and (v) their combination, where each of the n candidates is refined and the best final candidate is selected. We use $n = 8$ and $T = 3$. Among accepted candidates one is chosen uniformly at random; if none is accepted, one candidate is chosen at random.

Computation. For a procedure that yields, per candidate, the probabilities a_c (accepted and correct), a_w (accepted and incorrect), and r_c (rejected and correct), the success probability of selecting among n candidates is

$$P_n = \big(1 - (1-a)^n\big)\frac{a_c}{a} + (1-a)^n \frac{r_c}{1-a}, \qquad a = a_c + a_w. \tag{6}$$

For a single sample, $a_c = p\,\mathrm{TPR}$, $a_w = (1-p)\,\mathrm{FPR}$, and $r_c = p\,(1-\mathrm{TPR})$. With the feedback loop, let $f = a_c + a_w$ and $f_q = q\,\mathrm{TPR} + (1-q)\,\mathrm{FPR}$. A candidate that is rejected is refined up to

T times, which gives

$$
\begin{aligned}
a_c^{(T)} &= p\,\mathrm{TPR} + (1-f)\sum_{t=1}^{T}(1-f_q)^{t-1}\,q\,\mathrm{TPR},\\
a_w^{(T)} &= (1-p)\,\mathrm{FPR} + (1-f)\sum_{t=1}^{T}(1-f_q)^{t-1}(1-q)\,\mathrm{FPR},\\
r_c^{(T)} &= p\,(1-\mathrm{TPR})(1-f_q)^{T}.
\end{aligned}
\tag{7}
$$

The feedback loop alone succeeds with probability $a_c^{(T)} + r_c^{(T)}$ (it returns the initial output if no refinement is accepted), and the combined method is obtained by inserting Eq. (7) into Eq. (6). All values were computed in closed form and verified with Monte Carlo simulation (3×10^5 trials), with differences below 0.1 percentage points.

4.2 Main Results

Table 2 reports the success probability of each method in the three scenarios.

Table 2: Success probability (%) under the analytical model ($p = 0.40$, $q = 0.50$, $n = 8$, $T = 3$). Best results in bold.

Method	A	B	C
Single sample	40.0	40.0	40.0
Random selection ($n = 8$)	40.0	40.0	40.0
Best-of-n ($n = 8$)	95.2	84.7	69.1
Feedback loop ($T = 3$)	89.7	82.2	70.4
Combined	**97.5**	**88.1**	**73.2**

Random selection performs exactly like a single sample, so none of the improvement comes from the additional samples alone: it comes from the reward signal. Best-of-n raises the success probability from 40.0% to 95.2% under the verifiable reward, but only to 69.1% under the model-based reward. The feedback loop is weaker than best-of-n in Scenarios A and B, but stronger in Scenario C (70.4% vs. 69.1%). The combined method is the best in all three scenarios, improving over best-of-n by 2.3, 3.4, and 4.1 percentage points, respectively.

4.3 Effect of the Number of Candidates

Table 3 shows how the success probability changes with n.

Table 3: Success probability (%) as a function of the number of candidates n ($T = 3$ for the combined method).

Sc.	Method	n=1	2	4	8	16	32
A	Best-of-n	40.0	62.3	84.1	95.2	96.9	96.9
A	Combined	89.7	96.9	97.5	97.5	97.5	97.5
B	Best-of-n	40.0	59.2	76.8	84.7	85.7	85.7
B	Combined	82.2	87.6	88.1	88.1	88.1	88.1
C	Best-of-n	40.0	54.4	65.5	69.1	69.4	69.4
C	Combined	70.4	73.1	73.2	73.2	73.2	73.2

Performance rises quickly with n and then saturates. The ceiling is set by the precision of the reward, $p\,\mathrm{TPR}/\big(p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR}\big)$, which equals 96.9%, 85.7%, and 69.4% in Scenarios A, B, and C. Once the ceiling is reached, additional candidates add computation without improving quality. Refinement raises the ceiling itself: the combined method reaches higher values with fewer candidates, for example 96.9% at $n = 2$ in Scenario A, which best-of-n matches only at $n = 16$.

4.4 Discussion

The results show three main effects. First, the reward signal is the source of the gain, since random selection gives none. Second, the benefit depends strongly on reward quality: for the same n, moving from a verifiable to a noisy model-based reward reduces the success probability from 95.2% to 69.1%, and no number of candidates compensates for this loss because success saturates at the reward's precision. This supports the design principles in Section 2. Third, refinement and selection are complementary: refinement generates new candidates instead of only filtering existing ones, which is why the feedback loop is competitive under the noisiest reward, and combining both gives the best results in every scenario.

These gains come at a cost. Best-of-n requires n generations, the feedback loop up to $T + 1$ sequential ones, and the combined method up to $n(T + 1) = 32$. Since these results come from an idealized model with assumed parameters, they describe the qualitative behavior of reward-guided inference and not the performance of a specific LLM.

5 Ablation

To identify which components drive the improvements in Section 4, we vary one factor at a time while keeping all others fixed. All ablations use the analytical model of Section 4 with the same parameters ($p = 0.40$, $q = 0.50$, $n = 8$, $T = 3$) and the same three reward scenarios A, B, and C (Table 1).

5.1 Ablation Factors

Selection mechanism. We compare the full method against removing the reward from the selection step (a random candidate is chosen among the n refined candidates) and against removing the feedback loop (best-of-n only), to measure the contribution of each mechanism in Section 3.

Feedback content. In the feedback loop (Eq. (5)), detailed feedback f_t tells the model what to fix, giving a refinement success probability $q = 0.50$. With scalar-only feedback, the model receives only the score $R(x, y_t)$ and no information about the error, so a refinement is no better than a fresh sample: $q = p = 0.40$.

Reward type. We compare the verifiable (A), rule-based (B), and model-based (C) rewards with a composite reward (Eq. (2)) that accepts an output only if both the rule-based and the model-based reward accept it. Assuming independent errors, its rates are $\mathrm{TPR} = 0.90 \times 0.85 = 0.765$ and $\mathrm{FPR} = 0.10 \times 0.25 = 0.025$.

Number of candidates and iterations. We vary $n \in \{1, 2, 4, 8, 16, 32\}$ for best-of-n and the combined method (Table 3), and the maximum number of refinement iterations $T \in \{0, 1, 2, 3, 5\}$ (Table 6).

5.2 Results

Table 4 reports the effect of removing or changing each component of the full method. Δ is the average change over the three scenarios relative to the full method.

Table 4: Ablation of the method components: success probability (%) in Scenarios A–C.

Variant	A	B	C	Δ
Full method	97.5	88.1	73.2	–
w/o reward in selection	89.7	82.2	70.4	−5.5
w/o feedback loop	95.2	84.7	69.1	−3.3
Scalar-only feedback	96.9	85.7	69.4	−2.3

Table 5 compares reward types.

Table 5: Effect of the reward type: success probability (%) with $n = 8$ and $T = 3$.

Reward	Best-of-n	Loop	Combined
Verifiable (A)	95.2	89.7	97.5
Rule-based (B)	84.7	82.2	88.1
Model-based (C)	69.1	70.4	73.2
Composite (B+C)	91.6	83.9	96.3

Table 6 reports the effect of the number of refinement iterations T ($n = 8$).

Table 6: Success probability (%) as a function of the number of iterations T ($n = 8$ for the combined method). Sc. = scenario.

Sc.	Method	T=0	1	2	3	5
A	Loop	40.0	67.9	82.3	89.7	95.5
A	Combined	95.2	97.4	97.5	97.5	97.5
B	Loop	40.0	64.1	76.2	82.2	86.7
B	Combined	84.7	87.5	87.9	88.1	88.2
C	Loop	40.0	58.4	66.6	70.4	72.8
C	Combined	69.1	72.3	73.0	73.2	73.4

5.3 Analysis

Reward in selection. Removing the reward from selection is the most costly change (average −5.5 points). Without it, choosing randomly among independently refined candidates is equivalent to using a single refined candidate, so the result equals the feedback loop alone. Removing the reward costs 7.8, 5.9, and 2.8 points in Scenarios A, B, and C, whereas removing the loop costs 2.3, 3.4, and 4.1 points. Selection therefore matters most when the reward is reliable, and refinement becomes more important as the reward gets noisier.

Feedback content. With scalar-only feedback, refinement reduces to resampling, so the combined method equals best-of-32 ($n(T+1) = 32$ draws) and reaches exactly the precision ceilings of 96.9%, 85.7%, and 69.4%. Detailed feedback adds 0.6, 2.4, and 3.8 points, so its value grows as the reward gets noisier.

Reward type. Verifiable rewards give the highest success, and the model-based reward the lowest, in every method. The composite reward improves over both of its components (best-of-n: 91.6% vs. 84.7% and 69.1%) because requiring agreement lowers the false-positive rate, while remaining below the verifiable reward. This gain assumes independent errors; correlated errors would reduce it.

Candidates and iterations. Success increases with n and T with diminishing returns. The loop alone depends strongly on T (in Scenario A it rises from 40.0% to 95.5% at $T = 5$), whereas the combined method is almost saturated by $T = 2$ because many candidates are already available. The model does not reproduce the decline caused by over-optimization discussed in Section 3.3, because the false-positive rate is held fixed; capturing it would require a rate that grows with optimization pressure.

6 Conclusion

This paper studied how reward functions can be used to improve the output quality of LLMs. We formalized a reward function as a scalar mapping from a prompt and an output to a quality score, categorized the main reward families (rule-based, verifiable, and model-based), and showed how the resulting signal can be applied at inference time through best-of-n selection and generate–score–refine loops, without modifying the model weights.

Our experiments across [mathematical reasoning, code generation, and instruction following] showed that reward-guided inference [improves over single-sample generation by XX% on average], and the ablation study indicated that [the reward signal, not the additional sampling, is the main source of the gain]. The results also confirmed that the benefit depends on the quality of the reward: verifiable rewards gave the most reliable improvements, while model-based rewards were more sensitive to noise and exploitation.

Practical guidelines. Based on these findings, we recommend that practitioners (i) prefer verifiable or rule-based rewards whenever the task allows, (ii) combine several independent criteria to reduce exploitation, (iii) return detailed feedback, not only scores, in refinement loops, and (iv) tune the number of candidates and iterations instead of maximizing them, since over-optimization can reduce true quality.

Limitations. This study has several limitations. The experiments cover [a limited set of tasks and models], and the conclusions may not generalize to other domains or model families. Model-based rewards were evaluated using [an LLM judge] whose own biases were not fully controlled. In addition, inference-time methods increase computational cost linearly with the number of candidates, which may be prohibitive in latency-sensitive applications.

Future work. Future research could extend this work to open-ended tasks where no verifiable reward exists, study methods for detecting reward hacking automatically, explore adaptive strategies that choose n and the number of iterations per prompt, and compare inference-time reward use with training-time approaches such as reinforcement learning from feedback.

References

[1] Ouyang, L., et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 2022.

[2] Schulman, J., et al. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[3] T. Brown, et al., Language models are few-shot learners, *Advances in Neural Information Processing Systems* (2020).

[4] P. Christiano, et al., Deep reinforcement learning from human preferences, *Advances in Neural Information Processing Systems* (2017).

[5] Y. Bai, et al., Constitutional AI: Harmlessness from AI feedback, *arXiv preprint arXiv:2212.08073* (2022).

[6] K. Cobbe, et al., Training verifiers to solve math word problems, *arXiv preprint arXiv:2110.14168* (2021).

[7] A. Madaan, et al., Self-Refine: Iterative refinement with self-feedback, *Advances in Neural Information Processing Systems* (2023).

[8] L. Gao, J. Schulman, J. Hilton, Scaling laws for reward model overoptimization, *International Conference on Machine Learning* (2023).

[9] H. Lightman, et al., Let's verify step by step, *arXiv preprint arXiv:2305.20050* (2023).

[10] D. Amodei, et al., Concrete problems in AI safety, *arXiv preprint arXiv:1606.06565* (2016).

[11] R. Nakano, et al., WebGPT: Browser-assisted question-answering with human feedback, *arXiv preprint arXiv:2112.09332* (2021).

[12] C. Snell, J. Lee, K. Xu, A. Kumar, Scaling LLM test-time compute optimally can be more effective than scaling model parameters, *arXiv preprint arXiv:2408.03314* (2024).

[13] N. Shinn, et al., Reflexion: Language agents with verbal reinforcement learning, *Advances in Neural Information Processing Systems* (2023).

[14] M. Chen, et al., Evaluating large language models trained on code, *arXiv preprint arXiv:2107.03374* (2021).

[15] J. Zhou, et al., Instruction-following evaluation for large language models, *arXiv preprint arXiv:2311.07911* (2023).